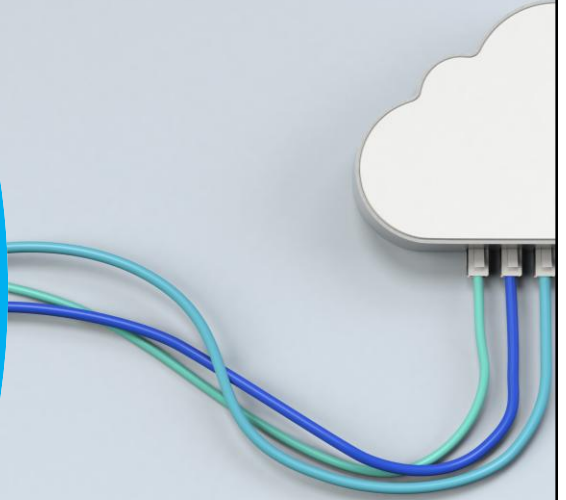


Sitzungsschicht

NETZWERKTECHNIK / SEMESTER 1 UND 2



1

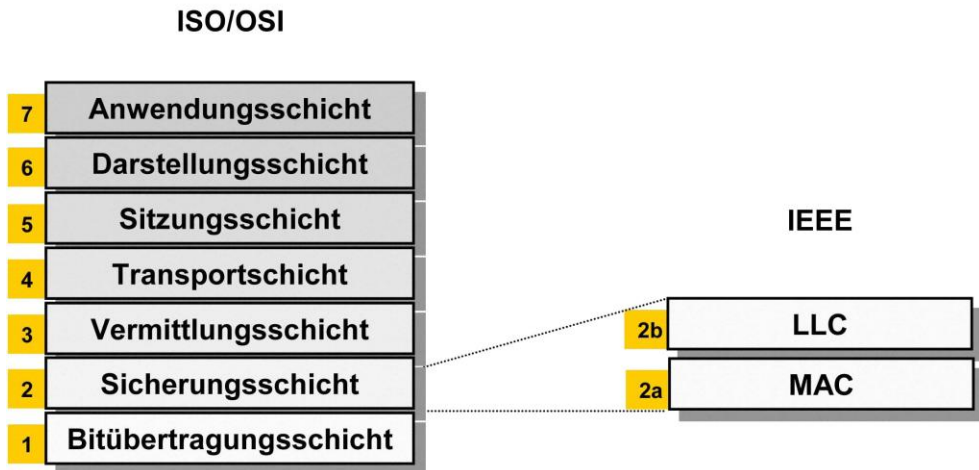
AGENDA

- 01 SITZUNGSSCHICHT
- 02 REALISIERUNG DER FUNKTIONALITÄT DER SITZUNGSSCHICHT

2

NETZWERKTECHNIK / SEMESTER 1 und 2

Einordnung im ISO/OSI Modell



[Quelle: https://www.airnet.de/cr1-gfe/de/html/TrFddIIEEE802_learningObject6.xml] -
 letzter Abruf 21.07.2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

3

3

01

Sitzungsschicht

4

Einordnung im ISO/OSI-Modell

Modell und Realität

- Die Standards der ITU-Serie X.200 beschreiben Dienste der sieben OSI-Schichten sowie Protokolle zur Erbringung dieser Dienste.
- Die in diesen Standards vorgenommene Strukturierung ist nützlich.
- Etliche OSI-Protokolle haben in der Praxis kaum Bedeutung.
- Oft ist keine strikte Trennung zwischen Sitzungs-, Darstellungsschicht und Anwendungsschicht möglich.
- Im Internet Model (RFC 1122) werden alle diese Funktionen der Anwendungsschicht zugeordnet.

Im Folgenden werden wir

- die Aufgaben der Sitzungs- und Darstellungsschicht erläutern,
- beispielhaft einige Protokolle kennenlernen, deren Funktionen den Schichten 5, 6 und 7 zugeordnet werden können,
- sowie wichtige Protokolle der Anwendungsschicht erläutern.

Sitzungsschicht

Gemäß Standard X.200 sind Dienste der Sitzungsschicht (engl. Session Layer) entweder verbindungsorientiert oder verbindungslos:

- **verbindungsorientiert (engl. „connection-oriented“)**
 - Es wird eine Verbindung zwischen den Kommunikationspartnern aufgebaut.
 - Die Verbindung bleibt dabei über die Dauer einzelner Transfers (oder Verbindungen) der Transportschicht hinweg bestehen.
 - Wie bei TCP-Verbindungen kann auch hier zwischen den Phasen **Verbindungsaufbau, Datentransfer und Verbindungsabbau** unterschieden werden.
- **verbindungslos (engl. „connection-less“)**
 - Daten werden im Wesentlichen nur an die Transportschicht durchgereicht.
 - Es wird keine Verbindung aufgebaut und kein Zustand zwischen den Kommunikationspartnern gehalten.

Hinweis: Eine Verbindung der Sitzungsschicht (Session) ist nicht gleichbedeutend mit einer Verbindung der Transportschicht. Eine Session kann beispielsweise nacheinander mehrere TCP-Verbindungen beinhalten oder Funktionen wie Bestätigungen unserem UDP-Chat hinzufügen, welche im Fall des TCP-Chats automatisch durch die Transportschicht erbracht werden.

Dienste der Sitzungsschicht

Eine Session beschreibt die Kommunikation zwischen mindestens zwei Teilnehmern mit definiertem Anfang und Ende sowie sich daraus ergebender Dauer.

Um für die dienstnehmende Schicht (Darstellungsschicht) eine Dialogführung zu ermöglichen, müssen gegebenenfalls mehrere Transportschicht-Verbindungen verwendet und kontrolliert werden. Dies kann auch die Behandlung abgebrochener und wiederaufgenommener TCP-Verbindungen beinhalten.

Im verbindungsorientierten Modus werden verschiedene Dienste angeboten:

- **Aufbau** und **Abbau** von Sessions,
- normaler und **beschleunigter Datentransfer**,
- **Token-Management** zur Koordination der Teilnehmer,
- **Synchronisation** und Resynchronisation,
- **Fehlermeldungen** und Aktivitätsmanagement, sowie
- **Erhaltung und Wiederaufnahme** von Sessions nach Verbindungsabbrüchen.

02

Realisierung der Funktionalität der Sitzungsschicht

Realisierung der Funktionalität der Sitzungsschicht am Beispiel von HTTP (Hyper Text Transfer Protocol)

- HTTP in seiner ursprünglichen Form ist zunächst zustandslos.
- Zwischen unterschiedlichen Anfragen und Antworten besteht daher zunächst kein Zusammenhang.
- **Cookies** ermöglichen es, dass eine Sitzung über mehrere Anfragen und Antworten, Interaktionen und TCP-Verbindungen hinweg bestehen bleibt.
- Cookies sind kleine Datenfragmente, welche von einem Webserver (bzw. einem damit verbundenen Anwendungsprozess) an einen Webclient übertragen und dort gespeichert werden können.
- Dadurch wird es möglich, zeitlich getrennte oder von unterschiedlichen Adressen stammende Anfragen einem bestimmten Client und damit einem bestimmten Nutzer zuordnen zu können.
- HTTP wird üblicherweise der Anwendungsschicht (Schicht 7) zugeordnet, beinhaltet aber auch Funktionen der Darstellungs- und Sitzungsschicht (Schichten 6/5).

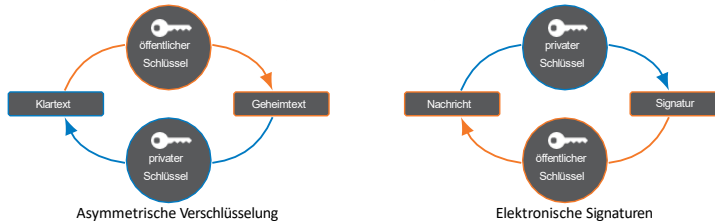
Hinweis: Bei HTTP handelt es sich um ein Protokoll der Anwendungsschicht.

Realisierung der Funktionalität der Sitzungsschicht am Beispiel von TLS (Transport Layer Security)

- TLS ist ein Protokoll zur **sicheren Übertragung von Daten über einen verbindungsorientierten Transportdienst**.
- Es ist die Grundlage u.a. für HTTPS.
- Es bietet unter anderem
 - **Authentifizierung** („ist mein Gegenüber der, für den ich ihn halte“),
 - **Integritätsschutz** (Schutz vor Manipulation von Daten) und
 - **Verschlüsselung** (Vertraulichkeit, d. h. Schutz vor unberechtigtem Mitlesen).
- Es gibt unzählige kryptographische Verfahren, viele sind unsicher. **Client und Server müssen sich auf die bestmöglichen Parameter einigen, die beide implementieren (Kompatibilität vs. Sicherheit).**
- Sitzungen können mit Hilfe von **Session-IDs** über mehrere TCP-Verbindungen hinweg erhalten bleiben.
- Während die Funktionen zur Sitzungsverwaltung und -wiederaufnahme eher der Sitzungsschicht zugeordnet werden, gehören die Verschlüsselungsfunktionen zur Darstellungsschicht.

TLS (Transport Layer Security) - Authentifizierung durch X.509 Zertifikate

- Bei der **Public-Key-Kryptographie** gibt es ein Schlüsselpaar: einen geheimen **Private Key** und einen öffentlichen **Public Key**. Dies ermöglicht sowohl Verschlüsselung als auch Signaturen.
- X.509-Zertifikate sind standardisierte Public Keys**, die mit zusätzlichen Informationen versehen und durch Signaturen kryptographisch abgesichert sind.
- Problem: Ist mein Gegenüber wirklich der, für den ich ihn halte?**
 - Kennt und vertraut man dem Zertifikat des Gegenübers, kann dieser authentifiziert werden indem er mit seinem Private Key die aktuelle Sitzung signiert.
 - Dies funktioniert nur, solange der **Private Key geheim bleibt!**



Data		
Version	Serial Number	Signature Algorithm
3	1	SHA-256 with RSA
Subject (Inhaber)		
CN=net.in.tum.de		
Issuer (Aussteller)		
C=US, O=Let's Encrypt, CN=R3		
Valid Not Before		Valid Not After
17. Januar 2024 05:02		16. April 2024 05:02
X509v3 Extension: Alternative Names		
*.net.cit.tum.de, *.net.in.tum.de, net.cit.tum.de, net.in.tum.de		
Subject Public Key Info		
RSA 2048: AA:BB:12:17:5F:57:47:1D:43:72:15:DB [...]		
Signature		
F1:BA:2C:DE:E6:03:F5:91:F3:EF:B8:17:69:AF:C3 [...]		

Eine globale X.509 Public-Key-Infrastruktur (PKI)

Es ist praktisch unmöglich, jedes Zertifikat einzeln zu kennen und individuell zu vertrauen.

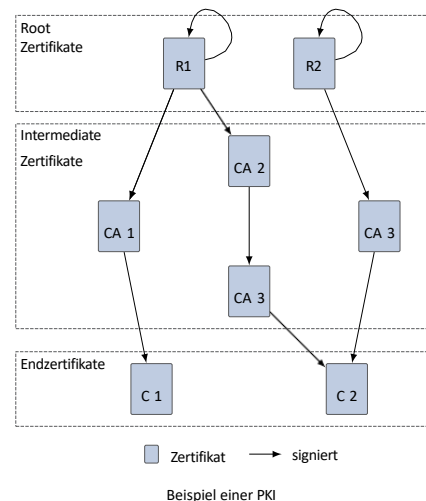
Idee: Vertraut man einem Zertifikat, kann man auch allen Zertifikaten vertrauen, die von diesem signiert wurden.

Zertifikate können andere Zertifikate signieren und somit Vertrauen weiterreichen, es entsteht eine Public-Key-Infrastruktur (PKI).

Man unterscheidet:

- Root / Wurzel Zertifikate**, diese sind auf dem Betriebssystem oder im Browser vorinstalliert und durch sich selbst signiert.
- Intermediate / CA Zertifikate** gehören den zahlreichen **Certificate Authorities (CAs)**, die weitere Zertifikate ausstellen dürfen.
- End- / Leaf Zertifikate** dienen der Authentifizierung, dürfen keine weiteren Zertifikate ausstellen.

Ein Zertifikat ist gültig, wenn eine **Signaturkette (Chain of Trust)** bis zu einem Root-Zertifikat besteht.



Der TLS Handshake

- Das Herzstück von TLS ist der initiale **Handshake** (<https://tls13.xargs.org/>), bei dem die Authentifizierung erfolgt und Parameter ausgehandelt werden. Können sich die Parteien nicht einigen, wird der Handshake abgebrochen. Wichtige Punkte:
- Der Algorithmus für den **Schlüsselaustausch** und der gemeinsamen geheimen Schlüssel,
- Das **Signaturverfahren**, um die Zertifikate zu prüfen (und die konkreten Zertifikate), und
- Die Algorithmen für die **symmetrische Verschlüsselung**.
- TLS nutzt **asymmetrische** Kryptographie, um einen gemeinsamen geheimen Schlüssel auszuhandeln. Für die eigentliche Datenübertragung wird jedoch eine **symmetrische** Verschlüsselung verwendet, da diese wesentlich effizienter ist.

Hinweis: Internet Messungen ergaben, dass manche Middleboxen (z.B. Router) unbekannte TLS Versionen blockieren. Deshalb hat man TLS 1.3 so spezifiziert, dass es wie ein TLS 1.2 Handshake aussieht.

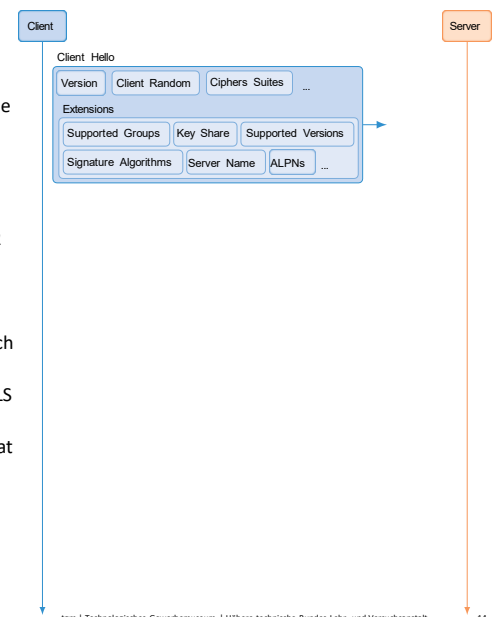
Der TLS Handshake

Client Hello

Der Start vom Handshake. Der Client schlägt die kryptographischen Algorithmen vor, die er unterstützt. Die Reihenfolge bestimmt die Präferenz.

Die Client Hello Nachricht enthält u.a.:

- Die **TLS Version** des Handshakes. In TLS 1.3 wird hier Version 1.2 versendet.
- Eine Liste an **Cipher Suites**. Dies sind eine spezifizierte Kombination aus kryptographischen Algorithmen. Alle TLS 1.3 ciphers gelten als sicher. Für TLS $\leq$ 1.2 gibt es auch unsichere ciphers, diese sollte man deaktivieren!
- Die Supported Groups Extension enthält eine Liste an **Algorithmen für den Schlüsselaustausch** (in $\leq$ 1.2 war dies Teil der cipher suit).
- In der **Key Share** Extension kann der Client den Schlüsselaustauschs opportunistisch vorberechnen.
- Erst die **Supported Versions Extension** verrät dem Server, dass es sich um einen TLS 1.3 Handshake handelt.
- Die unterstützten **Signatur Algorithmen**. Dies hilft dem Server das richtige Zertifikat auszuwählen.
- Der **Server Name** enthält die angefragten Domain. Wenn ein Server mehrere Domains bedient, kann er dadurch das richtige Zertifikat auswählen.
- ALPNs (Application Layer Protocol Negotiations)** enthalten eine Liste an **unterstützten Layer 7 Protokollen**, z.B. HTTP/1 oder HTTP/2.



Der TLS Handshake

2-RTT Fallback

- Wenn der Server mit dem Key Share des Client einverstanden ist, wird dieser Schritt übersprungen.
- Ansonsten informiert er den Client über den passenden Algorithmus durch eine **Hello Retry Request** Nachricht.
- Der Client kann nun ein **neues Client Hello generieren**, mit dem passenden Key Share.

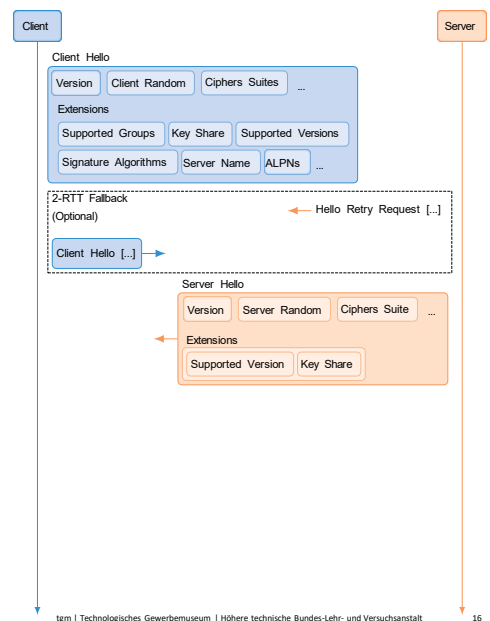
Hinweis: Dieser Schritt passiert in der Regel nur selten, da es aktuell nur wenige Algorithmen gibt, diese alle als sicher gelten, und es oft wichtiger ist Daten schnell an den Nutzer zu geben als die eigene Präferenz einzuhalten.



Der TLS Handshake

Server Hello

- Die Antwort auf das Client Hello.
- Der Server wählt hier jeweils einen der vorgeschlagenen Algorithmen aus.
- Da es unverschlüsselt ist, enthält es nur die nötigen Informationen für den Schlüsselaustausch (um das abhören **von Meta-Daten zu vermeiden**)



Der TLS 1.3 Handshake

Change Cipher Spec

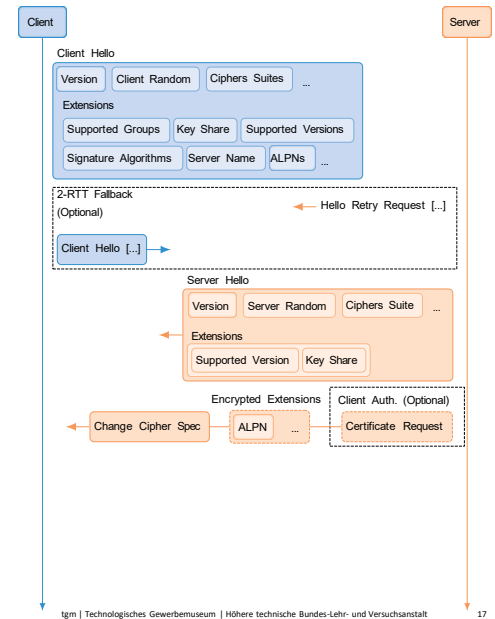
- Eine theoretisch unnötige Nachricht. Sie dient dazu Middleboxen vorzutäuschen, dass es sich um TLS 1.2 handelt.

Encrypted Extensions

- Beliebige weitere Informationen, wie das ausgewählte Layer 7 Protokoll.

Client Authentication

- Falls der Server auch den Client authentifizieren möchte, schickt er eine Certificate Request Nachricht. Für normale Webseiten unüblich.



Der TLS 1.3 Handshake

Certificate

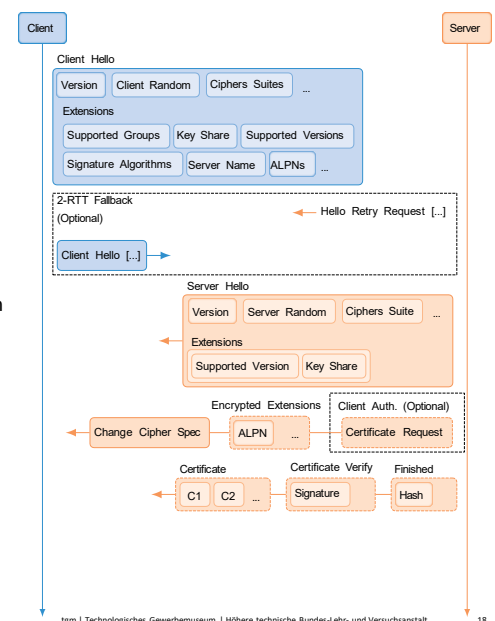
- Hier schickt der Server das eigene und alle Zertifikate, die der Client benötigt um die Chain of Trust zu berechnen.

Certificate Verify

- Diese Nachricht enthält eine Signatur von dem Server Zertifikat über ausgewählte Felder des bisherigen Handshakes. Dadurch beweist der Server den Besitz des private Keys zu dem Zertifikat.

Server Finished

- Diese Nachricht schließt den Handshake ab. Sie enthält einen abschließenden Hash über den bisherigen Handshake. Dies verhindert, dass ein Dritter den Handshake manipuliert hat.



Der TLS 1.3 Handshake

Client Change Cipher Spec

- Auch der Client schickt diese leere Nachricht

Client Authentication

- Falls der Server den Client authentifizieren möchte, muss dieser analog zum Server eine Certificate und Certificate Verify Nachricht schicken.
- Manche Unternehmen stellen z.B. eigene Zertifikat aus, um den Zugang zu internen Webseiten zu kontrollieren.

Client Finished

- Der Abschluss des Handshakes, ebenfalls mit einem Hash über den bisherigen Handshake
- Ab jetzt können Daten verschlüsselt ausgetauscht werden

